

1. Datos del Proyecto

Nombre del proyecto	Sistema de Control de Entrada y Salida de Vehículos en Taller Automotriz (SCEVTA)
Cliente / Solicitante	Edwin Steve Castillo Gómez
Sector	Taller Automotriz
Descripción	Sistema web para registrar y administrar el ingreso y salida de vehículos en un taller automotriz. Gestiona datos del cliente, características del vehículo, estado mecánico, servicios realizados, costos asociados e historial por vehículo.
Tipo de sistema	Microservicios
Plataforma	Web
Escala	< 100 usuarios (personal del taller: recepcionistas, mecánicos, administrador)
Presupuesto total	Q 20,000 (~USD 2,600). Infra mensual estimada: ~USD 182 / mes.
Entorno	Cloud — AWS
Metodología	Clásica — Modelo en Cascada (5 fases: Análisis, Diseño, Implementación, Pruebas, Mantenimiento)
Equipo	8 personas
BD provista	No indicada — la arquitectura define el motor

2. Requerimientos No Funcionales

Rendimiento

Tiempo de respuesta	< 2 segundos para operaciones CRUD estándar (registrar ingreso, consultar historial)
Latencia máxima (p95)	< 3,000 ms — sistema interno de un solo taller, tráfico muy bajo
Carga esperada	Máximo 10–20 usuarios simultáneos en hora punta del taller

Escalado	Vertical — una sola instancia es suficiente para toda la vida útil del sistema a esta escala
-----------------	--

Seguridad

Autenticación	JWT con refresh token — login por usuario del taller
Autorización	RBAC — roles: Administrador (acceso total), Recepcionista (clientes, vehículos, ingresos), Mecánico (servicios y estado del vehículo)
Seguridad en datos	TLS 1.2+ en tránsito. Campos de estado financiero del cliente cifrados en BD.
Respaldo de información	Backup automático diario (RDS). Retención: 7 días. RNF del análisis.

Usabilidad y confiabilidad

Interfaz	Fácil de usar para personal no técnico del taller (recepcionistas, mecánicos)
Acceso rápido	Búsqueda de vehículos por placa — resultado en < 1 segundo
Confiabilidad	Sin pérdida de datos — transacciones atómicas para ingreso/salida/servicios
Disponibilidad	99.5% — suficiente para un taller que opera en horario comercial

3. Arquitectura de Software

Estilo arquitectónico

Patrón elegido	Microservicios (un servicio por dominio de negocio)
Justificación	Indicación del ingeniero responsable del proyecto. Cada módulo del sistema (Clientes, Vehículos, Ingresos, Servicios, Facturación, Salida) se implementa como un servicio independiente con su propia base de datos. La comunicación entre servicios se realiza vía REST interno. Un API Gateway centraliza el acceso desde el frontend.

Módulos del sistema

Módulo	Responsabilidad	RF asociados
Clientes	Registro, consulta y actualización de datos del cliente. Estado financiero (solvente / deuda).	RF1
Vehículos	Registro de marca, modelo, placa, color, año. Asociación cliente-vehículo (1:N).	RF2
Estado del Vehículo	Inspección mecánica al ingreso: motor, transmisión, tren delantero/trasero, suspensión, estado general.	RF3
Ingreso	Registro de ingreso: fecha, motivo, kilometraje. Vincula cliente, vehículo y estado mecánico.	RF4
Servicios	Registro de servicios realizados durante la estadía: tipo, descripción, costo, fecha.	RF5
Facturación / Salida	Registro de salida: fecha, estado final, cálculo de total a pagar. Actualización estado financiero.	RF6
Historial	Consulta de historial completo por vehículo: ingresos previos, servicios y costos acumulados.	RF7
Autenticación	Login de usuario, gestión de JWT y control de acceso por rol (RBAC).	—

4. Stack Tecnológico

Capa	Tecnología	Versión	Justificación
Backend / API	NestJS	10.x	Framework estructurado sobre Node.js con módulos y DI nativos. Ideal para un sistema con 8 módulos bien delimitados (Clientes, Vehículos, Ingresos, etc.). Da convenciones claras para un equipo de 8 personas con metodología Clásica.
Frontend Web	Next.js	14	Routing integrado y layouts compartidos sin configuración adicional. Para un sistema interno con múltiples vistas de gestión operativa, Next.js organiza la navegación entre módulos de forma natural.
ORM	Prisma	5.x	Migraciones integradas y tipado fuerte. Las relaciones entre entidades (Cliente →

			Vehículo → Ingreso → Servicio) se modelan limpiamente con Prisma y son difíciles de romper accidentalmente.
Base de Datos	PostgreSQL	15	Relacional — las entidades tienen relaciones bien definidas con integridad referencial. Transacciones ACID para el registro de ingreso/salida/servicios en una sola operación atómica.
Autenticación	JWT (jsonwebtoken)	9.x	Stateless, fácil de implementar con RBAC de 3 roles. Sin necesidad de sesiones en servidor.
Generación PDF	PDFKit	—	Para generar reportes de historial y facturas de salida en PDF descargable. Sin dependencia de servicios externos.
Contenedores	Docker	24.x	Consistencia entre entornos. Docker Compose en producción es suficiente para una instancia única.
CI/CD	GitHub Actions	—	Pipeline automático: lint → test → build → deploy. Sin costo adicional.
Servidor web	NGINX	1.25	Proxy inverso, sirve el build de Next.js, termina TLS, redirige /api/* al backend NestJS.

5. Infraestructura

Componentes de red

Componente	Aplica	Herramienta	Notas
Proxy inverso / Servidor web	Sí	NGINX	Enruta tráfico al backend NestJS, sirve Next.js, termina TLS.
API Gateway	Sí	AWS API Gateway o NGINX como gateway	Punto de entrada único para todos los microservicios desde el frontend. Maneja routing, autenticación JWT centralizada y rate limiting.
Balanceador de carga	No	—	Con < 100 usuarios cada servicio corre en una sola instancia. Sin necesidad de ALB por servicio.

CDN	No	—	Sistema interno del taller, sin assets públicos pesados.
Cola de mensajes	No	—	Flujos síncronos entre servicios. Si en el futuro se añade notificaciones o reportes pesados, se evalúa.
WAF	No	—	Sistema interno, exposición pública mínima.
VPC / Red privada	Sí	AWS VPC	BD en subred privada — solo accesible desde el servidor de aplicación.

Servidores y capacidad

A < 100 usuarios en un taller, no se necesitan clústeres ni réplicas. Instancias únicas con backup automático.

Componente	Instancias	Spec / Tipo	Notas
API Gateway	1	EC2 t3.small o AWS API GW managed	Enruta peticiones del frontend al servicio correspondiente. Valida JWT antes de pasar la petición.
Svc. Clientes	1	EC2 t3.micro (NestJS + PostgreSQL propio)	Servicio independiente con su propio BD. Gestiona registro y estado financiero de clientes.
Svc. Vehículos	1	EC2 t3.micro (NestJS + PostgreSQL propio)	Gestiona marca, modelo, placa, año. Referencia clientes por ID.
Svc. Ingresos	1	EC2 t3.micro (NestJS + PostgreSQL propio)	Registra ingresos y estado mecánico del vehículo al momento de entrada.
Svc. Servicios	1	EC2 t3.micro (NestJS + PostgreSQL propio)	Registra servicios realizados (tipo, descripción, costo, fecha).
Svc. Facturación/Salida	1	EC2 t3.micro (NestJS + PostgreSQL propio)	Calcula total a pagar, registra salida y genera PDF con PDFKit.
Svc. Historial	1	EC2 t3.micro (NestJS + PostgreSQL propio)	Consulta agregada de ingresos y servicios previos por vehículo.
Staging (compartido)	1	EC2 t3.small	Un solo ambiente de staging con todos los servicios en contenedores para validar fases del Cascada.

6. Base de Datos

No hay insumo del área de BD en este ticket. La arquitectura define el modelo.

Motor	PostgreSQL 15
Tipo	Relacional
Justificación	Las 6 entidades del sistema (Cliente, Vehículo, EstadoVehículo, Ingreso, Servicio, Salida) tienen relaciones bien definidas: un cliente tiene N vehículos, un vehículo tiene N ingresos, un ingreso tiene N servicios. La integridad referencial y las transacciones ACID de PostgreSQL garantizan que un ingreso nunca quede sin vehículo asociado o un servicio sin ingreso.
Distribución por servicio	Cada microservicio tiene su propia instancia PostgreSQL. No comparten BD.
Svc. Clientes BD	Tabla: Cliente (id, nombre, teléfono, dirección, estado_financiero)
Svc. Vehículos BD	Tabla: Vehículo (id, cliente_id ref, marca, modelo, placa, color, año)
Svc. Ingresos BD	Tablas: Ingreso, EstadoVehículo (motor, transmisión, tren, suspensión, estado general)
Svc. Servicios BD	Tabla: Servicio (id, ingreso_id ref, tipo, descripción, costo, fecha)
Svc. Facturación BD	Tabla: Salida (id, ingreso_id ref, fecha_salida, estado_final, total_pagar)
Svc. Historial	Consulta agregada — llama a Svc. Ingresos y Svc. Servicios vía REST interno
Backup	RDS snapshot diario automático. Retención: 7 días. Cumple RNF de respaldo del análisis.
Migraciones	Prisma Migrate — cada fase del Cascada genera su migración versionada
ORM	Prisma 5.x
Acceso	Solo desde EC2 dentro de la VPC. Puerto 5432 no expuesto a internet.

7. Seguridad

Área	Implementación
Autenticación	JWT con expiración de 8 horas (jornada del taller) + refresh token.
Autorización	RBAC — 3 roles: Administrador (todo), Recepcionista (clientes, vehículos, ingresos y salidas), Mecánico (registro de servicios y estado del vehículo únicamente).
Contraseñas	bcrypt con salt rounds: 12.
Gestión de secretos	Variables de entorno en EC2. En producción: AWS Secrets Manager para credenciales de BD y JWT secret.
HTTPS	TLS 1.2+ — certificado gestionado con AWS Certificate Manager (gratuito).
CORS	Solo el dominio del frontend del taller. Bloqueado para cualquier origen externo.
Validación de inputs	class-validator en NestJS (DTOs) en todos los endpoints.
Estado financiero	Acceso a campo deuda/solvente restringido al rol Administrador. Cifrado en BD.
Logs de auditoría	Registro de cambios en estado financiero, ingresos y salidas de vehículos con usuario y timestamp.
Subred privada	RDS en subred privada de la VPC — sin acceso directo desde internet.

8. Contenedores y Despliegue

Compatible con la metodología Clásica: cada fase genera un entregable que se valida en staging antes de pasar a producción.

Estrategia	Docker + Docker Compose en producción. Una instancia EC2 t3.small.
Registry	AWS ECR
Proxy	NGINX — termina TLS, sirve Next.js compilado, redirige /api/* a NestJS.
Health checks	GET /health → 200 OK. GET /ready → verifica conexión a PostgreSQL.
Deploy	GitHub Actions → build Docker → push ECR → SSH a EC2 → docker compose pull && up -d.

Ambientes	dev (equipo) → staging (EC2 t3.micro, validación de fase Cascada) → production (EC2 t3.small).
Rollback	Imagen anterior en ECR. Si falla el smoke test: docker pull imagen-anterior && compose up.
Alineación Cascada	Cada entrega de fase (Diseño, Implementación, Pruebas) se despliega en staging para validación formal antes de avanzar a la siguiente fase.

9. CI/CD

Plataforma	GitHub (repositorio privado)
Branching	GitHub Flow simplificado — main (producción), develop (integración de fase), feature/* (funcionalidades)
Herramienta CI/CD	GitHub Actions
Etapas del pipeline	Lint (ESLint) → Tests (Jest) → Build Docker → Push ECR → Deploy staging → Smoke test → Deploy producción (manual por fase Cascada)
Alineación Cascada	El deploy a producción es manual y requiere aprobación formal — esto se alinea con la metodología Clásica donde cada fase tiene aprobación antes de avanzar.
Testing	Jest — tests unitarios de la lógica de negocio (cálculo de costos, validación de estado del vehículo, historial).

10. Observabilidad

Área	Aplica	Herramienta	Descripción
Logging	Sí	Winston + CloudWatch Logs	Logs estructurados: errores, accesos, ingresos/salidas de vehículos. Retención 30 días.
Métricas	Sí	CloudWatch Metrics	CPU, memoria, disco de EC2 y RDS. Incluido sin costo adicional.
Alertas	Sí	CloudWatch Alarms	Alerta si CPU > 80% por 5 min o si el health check falla 3 veces. Notificación por email.

Tracing distribuido	Sí	AWS X-Ray	En microservicios el tracing es necesario para detectar en qué servicio se origina un error o latencia. X-Ray integra con EC2 y API Gateway sin configuración adicional compleja.
Dashboards	No	—	CloudWatch básico es suficiente a esta escala.

11. Costos Estimados de Infraestructura

Componente	Servicio	Tier / Tipo	USD / mes	Notas
API Gateway (EC2)	AWS EC2	t3.small	~\$17	1 instancia para el gateway
6 microservicios (EC2)	AWS EC2	t3.micro × 6	~\$48	~\$8/mes por servicio × 6 servicios
6 BDs de servicios (RDS)	AWS RDS	db.t3.micro × 6 PostgreSQL 15	~\$90	~\$15/mes por BD × 6 servicios
Staging compartido	AWS EC2	t3.small (todos en Docker)	~\$17	Apagar fuera de fases de prueba
Almacenamiento (EBS)	AWS EBS	20 GB gp3 × 8 instancias	~\$8	Para EC2 + RDS
X-Ray (tracing)	AWS X-Ray	< 100K trazas/mes estimado	~\$0.50	Prácticamente gratuito a esta escala
Red / Transferencia	AWS	< 5 GB / mes	~\$1	Tráfico interno entre servicios
Monitoring / Logs	CloudWatch	Basic monitoring	\$0	Incluido en EC2 y RDS
Certificado SSL	AWS ACM	Gestionado	\$0	Gratuito con ACM prod + staging

12. Decisiones Arquitectónicas

ID	Área	Opción Elegida	Alternativa	Justificación
ADR-01	Comunicación entre servicios	REST interno (HTTP/JSON)	<i>gRPC, mensajería (RabbitMQ)</i>	Para 6 servicios a esta escala, REST interno es suficiente, más simple de implementar y depurar. gRPC añade complejidad de contratos proto sin beneficio perceptible. Mensajería solo si aparecen flujos asíncronos reales.
ADR-02	Framework backend	NestJS	<i>Express puro, Go/Fiber</i>	NestJS da estructura modular nativa para los 8 módulos del sistema. Go/Fiber es superior en rendimiento, pero para < 100 usuarios no hay diferencia perceptible, y NestJS acelera el desarrollo en metodología Clásica con fases predecibles.
ADR-03	Framework frontend	Next.js	<i>React puro, Vue</i>	Routing integrado y layouts compartidos para las 8 vistas del sistema (una por módulo). Next.js elimina la necesidad de configurar React Router y maneja el estado de navegación entre módulos de forma declarativa.
ADR-04	Base de datos	PostgreSQL	<i>MySQL, SQLite, MongoDB</i>	Las entidades del taller tienen relaciones bien definidas con integridad referencial crítica (un servicio no puede existir sin un ingreso). Transacciones ACID para el flujo ingreso → servicios → salida. MySQL también serviría, pero PostgreSQL es más robusto. MongoDB no aplica para datos relacionales.

ADR-05	Comunicación sync entre servicios	REST interno (HTTP/JSON)	<i>RabbitMQ, Kafka, gRPC</i>	Los flujos entre servicios son síncronos y de bajo volumen (registrar ingreso llama a Svc. Vehículos para validar). REST es suficiente y más simple de depurar que mensajería asíncrona a esta escala.
ADR-06	Generación PDF local	PDFKit	<i>Servicio externo, Puppeteer</i>	Los reportes de historial y facturas de salida se generan localmente sin dependencias externas. PDFKit es liviano y suficiente para documentos de taller.

12. Diagrama de Arquitectura

